

Technical Audit & Modernization Plan: WKS Platform

Role: Senior Full-Stack Architect & Cloud DevOps Engineer
Date: April 5, 2026

1. Executive Summary

This report outlines the technical strategy and effort estimation for migrating the WKS Platform from its current MongoDB/Form.io/Docker-based stack to a high-security, SQL-based (Azure SQL), and Azure-hosted environment.

2. Refactoring Map: Module-by-Module Audit

Module	Current State	Modernization Action
case-engine (Lib)	Direct MongoDB calls & Hybrid JPA (JSON in TEXT).	Rewrite: Full relational normalization for CaseInstance (split into case_comments, case_documents, case_attributes tables). Replace BSON/Mongo logic with Spring Data JPA.
api-security (Lib)	Deprecated AccessDecisionVoter (Spring Security 5 style).	Update: Migrate to AuthorizationManager (Spring Security 6). Refactor JwksIssuerAuthenticationManagerResolver to use standard multi-tenancy.
case-portal (React)	Hardcoded styles; Deep Form.io dependency.	Refactor: Implement Design Tokens via MUI Theme object. Replace @formio/react with React Hook Form + Zod.
FormService.js	Form.io-specific logic (e.g., recordtype mapping).	Rewrite: Decouple from Form.io data structures. Move to a standard JSON-schema-based retrieval and validation.
storage-api	MinIO-dependent.	Update: Abstract storage layer to support Azure Blob Storage via Spring Cloud Azure.

3. The "Form.io Exit" Strategy

The current dependency on Form.io creates a "black box" for form state and rendering. To achieve high flexibility and security:

- Target Engine:** React Hook Form (RHF) + Zod + MUI.
- Schema Standard:** Transition form definitions from Form.io's proprietary JSON to **standard JSON Schema (Draft 7/2019-09)**.
- Implementation Path:**
 - Schema Converter:** Develop a utility to map existing Form.io definitions to JSON Schema.
 - Generic Renderer:** Use @rjsf/mui (React JSON Schema Form) or a custom RHF-based builder to dynamically generate MUI components from the schema.
 - Validation:** Use zod-to-json-schema and vice-versa to ensure client-side and server-side validation are synchronized without manual code duplication.

4. **Custom Components:** Map the current `recordtype` logic to a custom MUI Autocomplete component that fetches data from the backend.

4. Effort Estimation (Man-Weeks)

Pillar	Focus Area	Estimated Effort
Frontend	Form.io Replacement, Theming (Design Tokens), Redux cleanup.	5-7 weeks
Database	NoSQL to SQL Server Normalization, Data Migration scripts.	4-5 weeks
Backend	Spring Security 6 cleanup, Jakarta EE compliance, API refactoring.	3-4 weeks
DevOps	Azure Infrastructure (Terraform/Bicep), CI/CD Security Gates.	2-3 weeks
Total		14 - 19 weeks

5. Risk Matrix (MongoDB → SQL Server)

Risk	Impact	Mitigation Strategy
Data Integrity	High	Use a staging migration tool (e.g., Azure Data Factory) with rigorous validation of nested arrays vs. joined tables.
Dynamic Schema Loss	High	Use SQL Server <code>JSON</code> column types for strictly dynamic "user-defined" fields, while normalizing core business entities.
Performance	Medium	Implement proper indexing on <code>business_key</code> and <code>uid</code> . Use Hibernate's <code>@BatchSize</code> for the newly normalized collection tables.
Feature Regressions	Medium	Implement Visual Regression Testing for the new form engine to ensure parity with Form.io's layout.

6. Target Azure Architecture

- **Web Tier:** Azure App Service for `case-portal` (Nginx/SPA) and Java APIs.
- **Data Tier:** Azure SQL Database (Serverless or Provisioned) with Multi-AZ.
- **Security:** Azure Key Vault for secret management; Microsoft Entra ID (OIDC) replacing/integrating with Keycloak.
- **Storage:** Azure Blob Storage (replacing MinIO).
- **CI/CD:** GitHub Actions with **CodeQL** (SAST) and **OWASP ZAP** (DAST) integration.